

Android.bp语法和使用方法

1. Android.bp 文件是什么？

Android.bp 文件首先是 Android 系统的一种编译配置文件，是用来代替原来的 [Android.mk](#)文件的。在 Android7.0 以前，Android 都是使用 make 来组织各模块的编译，对应的编译配置文件就是 [Android.mk](#)。

在 Android7.0 开始Google 引入了 ninja 和 kati 来编译，为啥引入 ninja? 因为随着 Android 越来越庞大，module 越来越多，编译时间也越来越久，而使用 ninja 在编译的并发处理上较 make 有很大的提升。Ninja 的配置文件就是Android.bp，Android 系统使用 Blueprint 和 Soong 工具来解析 Android.bp 转换成 ninja文件。为了兼容老的 mk 配置文件，Android 当初也开发了 Kati 工具来转换 mk 文件生成ninja。目前 Android Q 里边，还是支持 [Android.mk](#) 方式的。相信在将来的版本中，会彻底让 mk 文件废弃，同时 Kati 也就淘汰了，只保留 bp 配置方式，所以我们要提前学习bp。

这里涉及到Ninja, kati, Soong, bp概念，接下来分别简单介绍一下。

1.1 Ninja

ninja是一个编译框架，会根据相应的ninja格式的配置文件进行编译，但是ninja文件一般不会手动修改，而是通过将Android.bp文件转换成ninja格文件来编译。

1.2 Android.bp

Android.bp的出现就是为了替换[Android.mk](#)文件。bp跟mk文件不同，它是纯粹的配置，没有分支、循环等流程控制，不能做算数逻辑运算。如果需要控制逻辑，那么只能通过Go语言编写。

1.3 Soong

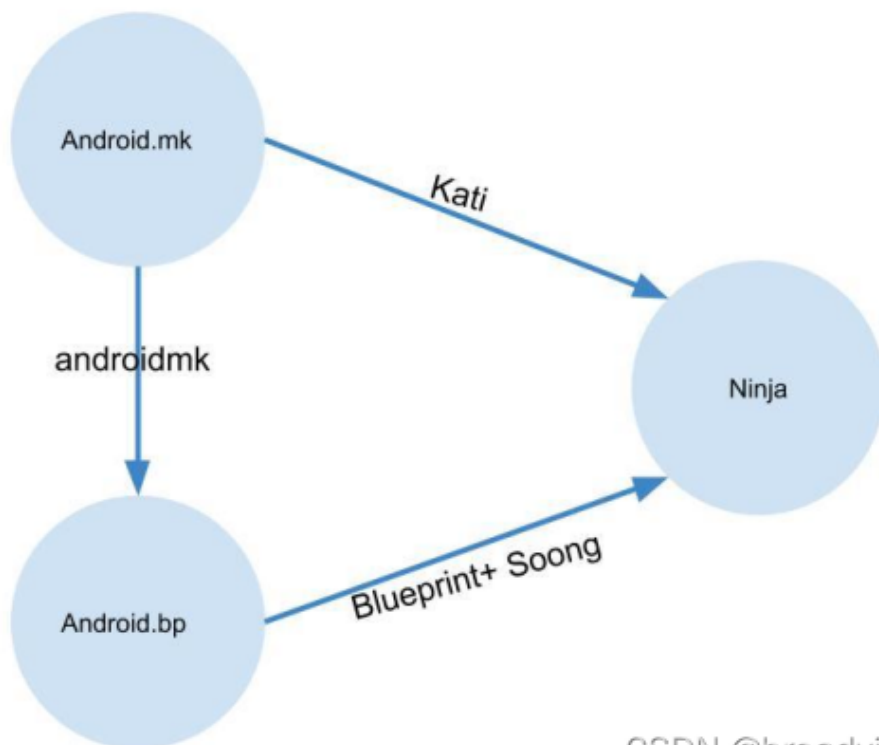
Soong类似于之前的Makefile编译系统的核心，负责提供Android.bp语义解析，并将之转换成Ninja文件。Soong还会编译生成一个androidmk命令，用于将[Android.mk](#)文件转换为Android.bp文件，不过这个转换功能仅限于没有分支、循环等流程控制的[Android.mk](#)才有效。

1.4 Blueprint

Blueprint是生成、解析Android.bp的工具，是Soong的一部分。Soong负责Android编译而设计的工具，而Blueprint只是解析文件格式，Soong解析内容的具体含义。Blueprint和Soong都是由Golang写的项目，从Android 7.0, prebuilts/go/目录下新增Golang所需的运行环境，在编译时使用。

1.5 Kati

kati是专为Android开发的一个基于Golang和C++的工具，主要功能是把Android中的[Android.mk](#)文件转换成Ninja文件。代码路径是build/kati/，编译后的产物是ckati。



2. 语法对应规则

我们可能已经习惯了[Android.mk](#) 中的语法，现在要变更为 [Android.bp](#)， 为了便于理解，可以找到源码， 查看[Android.mk](#) 与 [Android.bp](#) 语法对应规则：

源码位置： [/build/soong/androidmk/cmd/androidmk/android.go](#) 中， 这里我只粘贴一部分，完整代码请查看源文件。

```
var moduleTypes = map[string]string{
    "BUILD_SHARED_LIBRARY":    "cc_library_shared",
    "BUILD_STATIC_LIBRARY":    "cc_library_static",
    "BUILD_HOST_SHARED_LIBRARY": "cc_library_host_shared",
    "BUILD_HOST_STATIC_LIBRARY": "cc_library_host_static",
    "BUILD_HEADER_LIBRARY":    "cc_library_headers",
    "BUILD_EXECUTABLE":        "cc_binary",
    "BUILD_HOST_EXECUTABLE":    "cc_binary_host",
    "BUILD_NATIVE_TEST":       "cc_test",
    "BUILD_HOST_NATIVE_TEST":   "cc_test_host",
    "BUILD_NATIVE_BENCHMARK":   "cc_benchmark",
    "BUILD_HOST_NATIVE_BENCHMARK": "cc_benchmark_host",

    "BUILD_JAVA_LIBRARY":       "java_library_installable", // will be rewritten to java_library by bpfix
    "BUILD_STATIC_JAVA_LIBRARY": "java_library",
    "BUILD_HOST_JAVA_LIBRARY":   "java_library_host",
    "BUILD_HOST_DALVIK_JAVA_LIBRARY": "java_library_host_dalvik",
    "BUILD_PACKAGE":             "android_app",

    "BUILD_CTS_EXECUTABLE":      "cc_binary", // will be further massaged by bpfix depending
on the output path
    "BUILD_CTS_SUPPORT_PACKAGE": "cts_support_package", // will be rewritten to android_test by bpfix
    "BUILD_CTS_PACKAGE":         "cts_package", // will be rewritten to android_test by bpfix
    "BUILD_CTS_TARGET_JAVA_LIBRARY": "cts_target_java_library", // will be rewritten to java_library by bpfix
    "BUILD_CTS_HOST_JAVA_LIBRARY":  "cts_host_java_library", // will be rewritten to java_library_host by
bpfix
}

var prebuiltTypes = map[string]string{
    "SHARED_LIBRARIES": "cc_prebuilt_library_shared",
    "STATIC_LIBRARIES": "cc_prebuilt_library_static",
    "EXECUTABLES":      "cc_prebuilt_binary",
    "JAVA_LIBRARIES":    "java_import",
    "ETC":               "prebuilt_etc",
}
```

3. 如何把[Android.mk](#) 文件转换成 [Android.bp](#)

1. 在工程源码中：

1. source build/envsetup.sh

2. lunch xxx

3. make androidmk

生成androidmk转换工具，路径为：[/out/soong/host/linux-x86/bin/androidmk](#)

2. 直接把你想要转换的[Android.mk](#) 文件放置到此目录下，然后执行命令：

```
androidmk Android.mk > Android.bp
```

4. 语法讲解

为了便于理解，把[Android.mk](#) 和 [Android.bp](#) 的语法放在一起说明，更容易理解一点：

4.1 编译不同类型的模块

4.1.1编译成 Java 库

```
Android.mk
include $(BUILD_JAVA_LIBRARY)
```

```
Android.bp
java_library {
    .....
}
```

4. 1. 2编译成 Java 静态库

```
Android.mk
include $(BUILD_STATIC_JAVA_LIBRARY)
```

```
Android.bp
java_library_static {
    .....
}
```

4. 1. 3编译成 App 应用

```
Android.mk
include $(BUILD_PACKAGE)
```

```
Android.bp
android_app {
    .....
}
```

4. 1. 4 编译成Native动态库

```
Android.mk
include $(BUILD_SHARED_LIBRARY)
```

```
Android.bp
cc_library_shared {
    .....
}
```

4. 1. 5编译成 Native 静态库

```
Android.mk
include $(BUILD_STATIC_LIBRARY)
```

```
Android.bp
cc_library_static {
    .....
}
```

4. 1. 6编译成 Native 执行程序

```
Android.mk
include $(BUILD_EXECUTABLE)
```

```
Android.bp
cc_binary {
    .....
}
```

4. 1. 7编译成头文件库

```
Android.mk
include $(BUILD_HEADER_LIBRARY)

Android.bp
cc_library_headers {
    .....
}
```

4. 2文件路径

4. 2. 1本地头文件路径

```
Android.mk
LOCAL_C_INCLUDES :=

Android.bp
local_include_dirs: ["xxx", ...]
```

4. 2. 2导出的头文件路径

```
Android.mk
LOCAL_EXPORT_C_INCLUDE_DIRS :=

Android.bp
export_include_dirs: ["xxx", ...]
```

4. 2. 3资源文件路径

```
Android.mk
LOCAL_RESOURCE_DIR :=

Android.bp
resource_dirs: ["xxx", ...]
```

4. 3库依赖

4. 3. 1依赖的静态库

```
Android.mk
LOCAL_STATIC_LIBRARIES :=

Android.bp
static_libs: ["xxx", "xxx", ...]
```

4. 3. 2依赖的动态库

```
Android.mk
LOCAL_SHARED_LIBRARIES :=

Android.bp
shared_libs: ["xxx", "xxx", ...]
```

4. 3. 3依赖的头文件库

```
Android.mk
LOCAL_JAVA_LIBRARIES :=

Android.bp
header_libs: ["xxx", "xxx", ...]
```

4.3.4 依赖的 Java 库

```
Android.mk
LOCAL_STATIC_JAVA_LIBRARIES :=

Android.bp
static_libs: ["xxx", "xxx", ...]
```

4.4 安装到不同分区中

4.4.1 安装到vendor中

```
Android.mk
LOCAL_VENDOR_MODULE := true
    or
LOCAL_PROPRIETARY_MODULE := true

Android.bp
proprietary: true
    or
vendor: true
```

4.4.2 安装到product中

```
Android.mk
LOCAL_PRODUCT_MODULE := true

Android.bp
product_specific: true
```

4.4.3 安装到odm中

```
Android.mk
LOCAL_ODM_MODULE := true

Android.bp
device_specific: true
```

4.5 编译参数

4.5.1C flags

```
Android.mk
LOCAL_CFLAGS :=

Android.bp
cflags: ["xxx", "xxx", ...]
```

4.5.2Cpp flags

```
Android.mk
LOCAL_CPPFLAGS :=
```

```
Android.bp
cppflags: ["xxx", "xxx", ...]
```

4.5.3Java flags

```
Android.mk
LOCAL_JAVACFLAGS :=
```

```
Android.bp
javacflags: ["xxx", "xxx", ...]
```